Problem 4: Gated Linear Attention

GStar Bootcamp

Abstract

Add per-channel forgetting, a chunk-local gate scan, and gated linear-attention state/output kernels.

Starter file: `problem_4.py`. **Checker:** `python autograder.py -p4`. **Reading path:** Overview, P3. Next: P5.

Mathematical background: forgetting is a transport rule

Additive memory retains every write. Gating changes how an existing association survives to a later query. Let $\boldsymbol{\alpha}_t = e^{\boldsymbol{g}_t}$, with components in $(0, 1]$. Then

$$\mathbf{S}_t = \mathrm{Diag}(\boldsymbol{\alpha}_t)\mathbf{S}_{t-1} + \boldsymbol{k}_t\boldsymbol{v}_t^\top.$$

A write at time j is transported through gates $j+1$ through t; gate j acted before that write existed. This timing gives the relative-decay factors derived below. The gate is not a softmax weight: it modifies carried memory before any particular query reads it [1].

The diagonal transition preserves the separability of state rows. A scalar gate scales every row equally, while a channel gate permits different retention times. The homogeneous contribution $\mathrm{Diag}(\boldsymbol{\alpha}_t)\mathbf{S}_{t-1}$ cannot increase the Frobenius norm. This does not bound the entire state by itself, since new writes are still added. For a uniform scalar bound $\alpha_t \le \rho < 1$ and writes with norm at most M, repeated substitution gives

$$\|\mathbf{S}_t\|_F \le \rho^t \|\mathbf{S}_0\|_F + M\frac{1-\rho^t}{1-\rho}.$$

This bound explains the effect of sustained forgetting; a channel allowed to keep decay one need not satisfy a strict contraction bound.

The kernel task is to express the same transport in three places: old state reaching a query, local writes reaching a query, and local writes reaching the chunk exit. All three must agree on the gate interval, which is why the inclusive log-prefix scan is a shared mathematical primitive.

Before coding. Track one write for two tokens. It must have weight one at its own token and acquire only the later token's gate at the next read.

Forget before adding a new association

Linear attention keeps adding associations. GLA introduces a gate for each key channel [1]:

$$\mathbf{S}_t = \mathrm{Diag}(e^{\boldsymbol{g}_t})\mathbf{S}_{t-1} + \boldsymbol{k}_t\boldsymbol{v}_t^\top, \qquad \boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t, \tag{4.1}$$

where $\boldsymbol{g}_t \in \mathbb{R}^{D_k}$ and every component is nonpositive. The diagonal matrix scales *rows* of the $D_k \times D_v$ state. Implement this as elementwise broadcasting; do not allocate a diagonal matrix.

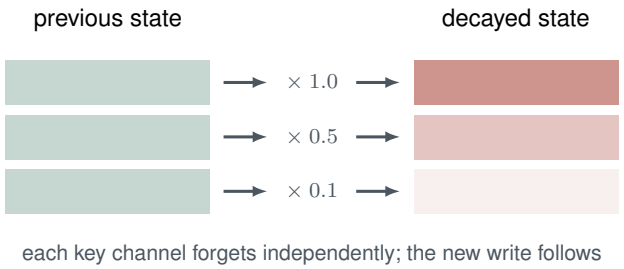


Figure 1. Channel gates scale state rows. The illustrated decays $(1, 0.5, 0.1)$ preserve one row, partially forget another, and strongly forget a third.

Log gates and cumulative decay

Products of decay factors become sums in log space. Define a prefix within one chunk:

$$\boldsymbol{b}_r = \sum_{j=1}^{r} \boldsymbol{g}_j. \tag{4.2}$$

The decay carrying an update from local position u to $r \geq u$ is

$$\prod_{j=u+1}^{r} e^{\boldsymbol{g}_j} = e^{\boldsymbol{b}_r - \boldsymbol{b}_u}.$$

The gate at u acts on the *previous* state; the write made at u is not immediately decayed by that same gate. Consequently the relative decay uses $\boldsymbol{b}_r - \boldsymbol{b}_u$, giving one on the diagonal.

Your task

Build the recurrent update, the chunk-local inclusive gate scan, and the chunk state/output kernels. A direct PyTorch gated-score baseline is provided temporarily; Problem 8 replaces it with your Triton score builder.

Problem 4: derive the gated chunk form

Let $\boldsymbol{b}_C$ be the final log-prefix row. Define channelwise transformed queries and keys:

$$Q^{\text{start}}_{rd} = Q_{rd} e^{b_{rd}}, \tag{4.3}$$

$$K^{\text{end}}_{ud} = K_{ud} e^{b_{Cd} - b_{ud}}. \tag{4.4}$$

The first carries the entry state to each query position. The second carries each local key write to the chunk end. With

$$F(\mathbf{Q}, \mathbf{K})_{ru} = \sum_{d=1}^{D_k} Q_{rd} K_{ud} e^{b_{rd} - b_{ud}}, \quad u \leq r, \tag{4.5}$$

and zeros above the diagonal, the chunk equations are

$$\mathbf{S}_{\text{out}} = \operatorname{Diag}(e^{\boldsymbol{b}_C}) \mathbf{S}_{\text{in}} + (\mathbf{K}^{\text{end}})^\top \mathbf{V}, \tag{4.6}$$

$$\mathbf{O} = \left[\mathbf{Q}^{\text{start}} \mathbf{S}_{\text{in}} + F(\mathbf{Q}, \mathbf{K}) \mathbf{V} \right]. \tag{4.7}$$

Worked gate example

For one channel, let $g_1 = \log(1/2)$ and $g_2 = \log(1/4)$. Then $b_1 = \log(1/2)$ and $b_2 = \log(1/8)$. The entry state reaches token 2 with decay $1/8$. A write at token 1 reaches token 2 with decay $e^{b_2 - b_1} = 1/4$. A write at token 2 has decay one.



entry contribution: $1/8$; write 1: $1/4$; write 2: 1

Figure 2. The local write is made after its token's gate. This ordering determines the prefix differences in the chunk formula.

Why the scan resets at a chunk boundary

Each chunk formula expresses decay relative to that chunk's entry state. The entry state already contains all earlier forgetting. Restart the log-prefix from zero in each chunk; including previous chunks' prefixes in the entry-state factor would apply earlier forgetting again.

The provided score baseline computes Eq. (4.5) directly in FP32, one chunk at a time. It does not use the optimized secondary-tile factorization. This keeps the early exercise focused on gates and state propagation.

Problem 4: implementation steps and checks

1. **Recurrent decay (25 points).** Load a channel gate vector, exponentiate it in FP32, and multiply state rows before adding the new outer product.
2. **Inclusive chunk scan (25 points).** Complete `_chunk_cumsum`. For input (C, B_K), scan along the token axis. Store FP32 log-prefixes and reset at every chunk.
3. **Chunk state and output (55 points).** Compute exit-decayed keys, decay the entry state by the final prefix, and implement Eq. (4.6). In the output kernel compute entry-decayed queries and combine their state read with the provided local score tile.
4. **Gate-axis explanation (20 points).** State which matrix axis is decayed and why that follows from the tensor contract.

```
# Gate-scan kernel: x has shape [CHUNK, BLOCK_K].
# TODO P4.A: inclusive prefix sum over local token positions.
# Store the result in FP32.

# Chunk-state kernel: k, v, prefix, and entry state are loaded.
# TODO P4.B: transform k from each write position to chunk end.
# TODO P4.C: decay entry state and add the local contributions.

# Output kernel: provided scores already obey causal masking.
# TODO P4.D: transform queries from chunk start to each row.
# TODO P4.E: combine the entry-state and local-value terms.
```

Verification

- **All gates zero:** recover linear attention from Problems 2 and 3.
- **Same gate for every channel:** recover a scalar-decay recurrence.
- **Unequal gates:** use $D_k = 2$ toy matrices to see different state rows decay independently.
- **Nonzero initial state, zero keys:** outputs read the decayed state; no new association is written.
- **Two chunks:** check the first prefix row of chunk 2 equals its first gate row, rather than a sequence-global sum.

Precision and finite values

Keep prefixes and exponent arguments in FP32. Gate prefixes are nonincreasing because $g \leq 0$. The entry factors e^{b_r} and exit factors $e^{b_C - b_u}$ are at most one. Problem 8 handles the numerical danger in separately factorizing relative gate scores.

Problem 4: deriving the complete transport of a write

Expand the gated recurrence inside a chunk

Write $\mathbf{G}_j = \mathrm{Diag}(e^{\boldsymbol{g}_j})$. Because all gate matrices are diagonal, their products commute with one another, and

$$\mathbf{G}_r \cdots \mathbf{G}_1 = \mathrm{Diag}(e^{\boldsymbol{b}_r}).$$

Repeated substitution in the additive recurrence yields

$$\mathbf{S}_r = \mathrm{Diag}(e^{\boldsymbol{b}_r})\mathbf{S}_{\mathrm{in}} + \sum_{j=1}^{r} \mathrm{Diag}(e^{\boldsymbol{b}_r - \boldsymbol{b}_j})\boldsymbol{k}_j \boldsymbol{v}_j^\top. \tag{4.8}$$

The diagonal factor for a write at j contains gates $j+1, \ldots, r$. It excludes gate j, because that gate was applied before the write. For $j = r$, the difference is zero and the current write has weight one.

Read and exit are two different transport destinations

Transpose Eq. (4.8) and multiply by $\boldsymbol{q}_r$:

$$\boldsymbol{o}_r = \mathbf{S}_{\mathrm{in}}^\top \mathrm{Diag}(e^{\boldsymbol{b}_r})\boldsymbol{q}_r + \sum_{j \le r} \big(\boldsymbol{q}_r^\top \mathrm{Diag}(e^{\boldsymbol{b}_r - \boldsymbol{b}_j})\boldsymbol{k}_j\big)\boldsymbol{v}_j.$$

The entry contribution becomes a query transformed from the chunk start to row r. The local coefficient is a channel-weighted inner product. For the exit memory, set $r = C$ instead; each write's key is transported to the chunk end. This is why queries use $e^{\boldsymbol{b}_r}$ while exit keys use $e^{\boldsymbol{b}_C - \boldsymbol{b}_j}$.

Channel decay as a relative-memory curve

With a constant channel log gate $g_d = -\lambda_d$, a write that is ℓ tokens old is weighted by $e^{-\lambda_d \ell}$. Its half-life is $\log(2)/\lambda_d$ when $\lambda_d > 0$. Different channels can retain different time scales without changing the matrix-state shape.

Problem 4: prefix axes and neutral padding

The scan is inclusive, local, and channel-wise

For a loaded gate tile of shape $C \times B_K$, scan axis 0. The first output row equals the first input gate row; row r equals the sum of rows 1 through r. Each channel accumulates independently. A scan over key channels would sum unrelated forget mechanisms and would not represent temporal decay.

At the next chunk, restart the prefix at its first local gate. The carried matrix state already summarizes earlier gates. With an inclusive prefix, current-token score decay is $e^{b_{rd} - b_{rd}} = 1$; an exclusive prefix would shift decay timing and fail the worked two-token example.

Why padded log gates must be zero

A padded gate of zero gives $e^0 = 1$, preserving the memory state. Padding with a negative gate silently forgets memory after the real final token, so trimmed outputs may look correct while returned final state is wrong. Zero key/value rows ensure padded tokens contribute no additive write.

For an incomplete chunk with real final position $m < C$, the final padded prefix equals $\boldsymbol{b}_m$. Thus $\boldsymbol{b}_C$ used by the exit formula is exactly the final real prefix, and the returned state is the memory after token m.

P4 score and output scaling in the actual starter

The temporary score builder receives `scale=scale`, so its local score buffer already includes `scale`. The output kernel also multiplies its query load by `scale` for the entry-state read. Both output terms are therefore scaled once. Add them directly; do not multiply their sum by `scale` again.

When replacing the baseline with P8's `gated_scores`, preserve this calling convention. A zero-gate integration test must match P3 even with a custom query scale and a nonzero initial state.

Checks that separate gate problems

Compare the scan output before running attention. Compare the state pass with supplied correct prefixes before debugging local scores. If output is wrong only for unequal channel gates, inspect gate broadcasting and channel-weighted scores. If the first chunk is correct but the next one fails, inspect entry-state timing and whether prefixes restarted locally.

References

[1] S. Yang et al. *Gated Linear Attention Transformers with Hardware-Efficient Training*. 2023/2024. https://arxiv.org/abs/2312.06635